ASSIGNMENT 3 – TASK 3

Resource Safety, Debugging and Robust Code

Programming in Java | CO3 | 7 Marks | Bloom's Level: K4 – Analyse

1. Problem interpretation

A retail chain reads several store files and writes a daily sales summary. The reported "Too many open files" error indicates that resources are not reliably closed. The report may also be incomplete when a failure occurs part-way through processing. The implementation compares explicit manual close() calls with try-with-resources.

2. Resources that must be closed

Every opened input reader must be closed. Any output writer or stream introduced for the report must also be closed. Each resource should have a clearly defined owner and a guaranteed cleanup path. In the supplied demonstration, TrackedReader wraps BufferedReader and implements AutoCloseable.

3. Manual-close failure and debugging

The manual version opens readers and stores them in a list. It closes them only after the complete reading loop. The deliberately missing third file causes an IOException after two readers have been opened. Control jumps to catch, so the close loop is skipped. The finally block executes, but it only reports the tracked count; it does not release those readers. The demonstration therefore reports two open readers.

```
open store1.csv -> openReaders = 1
open store2.csv -> openReaders = 2
open missing store3_missing.csv -> IOException
control jumps to catch
manual close loop is skipped
finally reports openReaders = 2
```

4. Try-with-resources implementation

Each reader is declared in a try-with-resources statement. Java closes the reader automatically when the try block exits, whether it exits normally or because an exception is thrown. Resources are closed in the reverse order of their declaration when multiple resources are declared in one statement. In this example, each file has its own resource scope, so the previous file is closed before the next is processed.

5. Comparison and correctness

For successful input, both implementations process store1.csv, store2.csv and store4.csv and produce a total of 500 across three stores. With the same missing-third-file input, the manual version leaves two tracked readers open, while try-with-resources returns the count to zero. The tracked count is a program-level resource proxy; it is not a direct operating-system file-descriptor measurement.

6. Logging and debugging evidence

The program logs each file name before opening it. The final logged name identifies the file being processed when failure occurred. In a debugger, place breakpoints at reader creation, the manual close loop and the catch block. Step through the first two opens, then observe that the missing file exception transfers control to catch without visiting the close loop. Inspect the counter to verify the leak.

7. Suppressed exceptions

If the try body throws an exception and closing the resource also throws, the exception from the try body remains primary and the close exception is attached as a suppressed exception. Call getSuppressed() on the primary exception to inspect these additional failures. Suppressed exceptions preserve cleanup diagnostics rather than replacing the original failure.

8. Joint engineering decision

Try-with-resources is not sufficient by itself when the application has incorrect parsing or validation, partial-write or transaction problems, insufficient disk space, concurrency issues, or failures that require retry/recovery. It guarantees resource closure; it does not guarantee that a report is complete, valid, atomic or recoverable. The application still needs input validation, logging, error policy, and safe output-file handling.

9. Alternative approaches and complexity

For f files with a total of B input bytes, reading and processing is O(B) time, plus O(f) file-open/close operations. With one reader opened at a time, try-with-resources keeps the number of simultaneously open input handles at O(1). The intentionally flawed manual version retains one handle per successfully opened file until the close loop, so at a failure after k files it may retain O(k), and in the worst case O(f), handles. A corrected manual implementation can close every reader in a finally block, but must carefully handle null/unopened resources and exceptions during close. No closing can retain up to O(f) handles and can exhaust the process or operating-system limit. The input processing space is O(1) beyond buffers if rows are processed as streams; this sample stores report lines in memory, which is O(r) for r output lines.

10. Conclusion

Try-with-resources is the preferred implementation because resource cleanup is tied to lexical scope and occurs on both normal and exceptional exits. It reduces the chance of resource leaks and simplifies code. It must be combined with validation, logging and report-integrity controls for a robust production report generator.

Execution evidence and screenshots should be captured from the student’s own Java environment before submission.